

## PDN: GeorgeKuttyRenjan\_Shone\_Project\_1 PDF report

### Prob 1: Request Nodes

How do you request 2 compute nodes to run a job?

```
#SBATCH --nodes=2
```

### Prob 1: Job Status

How do you find the status of a job after it's submitted?

```
squeue -u username
```

### Prob 1: Login Node

Why should you NOT run a job on the login node?

Can cause the Schooner login node to crash

### Prob 2: Explanation

Key differences between the stack and the heap.

The key differences between the stack and heap is that the stack is automatically managed by the system and is used for local variables, function parameters, and information such as return addresses. While heap memory is the responsibility of the programmer to manage using malloc and free, and is generally intended for dynamically sized data whose size we do not know at compile time. The stack has a limited size and follows a LIFO allocation last in first out. Sometimes the heap can grow much larger than the stack and it does not follow a strict allocation order. When memory is allocated on the stack it is automatically released when function returns, while when a heap memory stays until it is explicitly freed by the programmer. Generally, large data should not be stored on the stack because they can cause a overflow.

The variables `n_row1`, `n_col1`, and `n_row2` are stored on the stack because they are smaller scalar values used only in the main function. File pointers like `matFile`, `vecFile`, and `outputFile` are stored on the stack because they are local variables that are involved in system like resources. Temporary loop variables are stored on the stack because they have a short lifetime they are not generally used after the loop purpose. The matrix & vector array is stored on the heap because its size is dynamic depending on the runtime input and can be large. The output vector result is

stored on the heap because it must hold the computed result for all rows of the matrix. The output vector result is stored on the heap because it must hold all the computed result for all rows the matrix. Laslty, the heap memory allocated for these variables are released with the free() to prevent memory leaks.

### Prob 3: 3A

Expected speedup and efficiency.

Please compute the expected speedup and efficiency in the two tables below:

| Speedup | n = 100 | n = 200 | n = 400 |
|---------|---------|---------|---------|
| p = 8   | 5.83    | 7.09    | 7.62    |
| p = 16  | 8.33    | 12.24   | 14.36   |
| p = 32  | 10.07   | 18.79   | 25.52   |

| Efficiency | n = 100 | n = 200 | n = 400 |
|------------|---------|---------|---------|
| p = 8      | 0.73    | 0.89    | 0.95    |
| p = 16     | 0.52    | 0.77    | 0.90    |
| p = 32     | 0.31    | 0.59    | 0.80    |

### Problem 3B.

The actual parallel runtime of your algorithm was benchmarked using real-world datasets. The benchmarking results are provided below:

| Runtime (seconds) | n = 100 | n = 200 | n = 400 |
|-------------------|---------|---------|---------|
| serial            | 113     | 428     | 1638    |
| p = 8             | 19      | 62      | 223     |
| p = 16            | 14      | 36      | 116     |
| p = 32            | 11      | 23      | 63      |

Please compute the observed speedup and efficiency in the two tables below:

| Speedup | n = 100 | n = 200 | n = 400 |
|---------|---------|---------|---------|
| p = 8   | 5.95    | 6.9     | 7.35    |
| p = 16  | 8.07    | 11.89   | 14.12   |
| p = 32  | 10.27   | 18.61   | 26      |

| Efficiency | n = 100 | n = 200 | n = 400 |
|------------|---------|---------|---------|
| p = 8      | 0.74    | 0.86    | 0.92    |
| p = 16     | 0.50    | 0.74    | 0.88    |
| p = 32     | 0.32    | 0.58    | 0.81    |